

Experiment 3

Arman Imam

EEE3342C

25Fall 0011

9/24/25

Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically.

Objective:

The objective of Experiment 3 is to introduce the user on how Boolean simplification can be applied in a digital circuit. For part 1, the user will simplify two equations, as well as identify a product of sums and sums of products from a truth table. For part 2, the user will make a 3-input circuit, where the user is to make a Boolean equation based on the truth table to test in the design, as well as test it with a simplified Boolean equation. Part 3 asks for the same deliverables as part 2, but it involves 4 inputs. Part 4 asks the user to utilize K-maps for simplification for a Boolean equation derived from a truth table. Part 5 asks the user to apply their skills in a scenario, where they simplify a 4-input design based on requirements involving 1-based, 0-based, and don't care outputs for input numbers.

Design Steps

I began by creating 4 separate folders that account for parts 2-5. For part 1, I've simplified 2 equations given by the experiment brief. For parts beyond this point, I use sum of products to simplify the Boolean expression. For part 3, I created a 3 input, 1 output design given by the truth table from the brief. Here I recorded the RTL code for the design and a more simplified design, along with the waveform, schematics for before and after simplification, and the simplified Boolean equation with steps. For part 3, I recorded the same data, with the difference being a new truth table and 4 inputs. For part 4, I recorded the same data, but this time, I simplified the Boolean equation with kmaps, and I reflected it in the schematic and RTL code. For part 5, I played the role of physical design engineer, and created a design based on the requirements given: 4-inputs: A, B, C, D, 1-Based Outputs for Input Numbers:3, 7, 8, 9, 10, 13, 0-Based Outputs for Input Numbers:1, 2, 5, 6, 11, 15, and Don't Care Outputs for Inputs Numbers:4, 12, 14, 0. I recorded the same deliverables as part 3.

Part 1:

$$\text{EQ1: } AB'C + A'BC' + A'B'C$$

Simplified by distributive property and XOR law:

$$AB'C + A'(BC' + B'C)$$

$$AB'C + A'(B \oplus C)$$

$$\text{EQ2: } ABC + AB'C + A'BC' + A'B'C$$

Simplified by distributive property and inverse law:

$$AC(B + B') + A'BC' + A'B'C$$

$$AC + A'BC' + A'B'C$$

Truth table:

Inputs			Outputs
A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Note:

SOP -> output of 1	1	0
	A	A'
POS -> output of 0	1	0
	A'	A

$$\text{SOP: } A'B'C + AB'C' + AB'C + ABC$$

$$\text{POS: } (A+B+C)(A+B'+C)(A+B'+C')(A'+B'+C)$$

Part 2: 3-input Boolean Circuit Simplification using Laws

Inputs			Outputs
A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- Verilog RTL code

The source code involves 3 inputs(A,B,C), and 1 output(F). The code before simplification is the most maximized. The simplified equation was achieved with distributive property and inverse law. They each provide the same waveform, indicating that they're equal.

BEFORE:

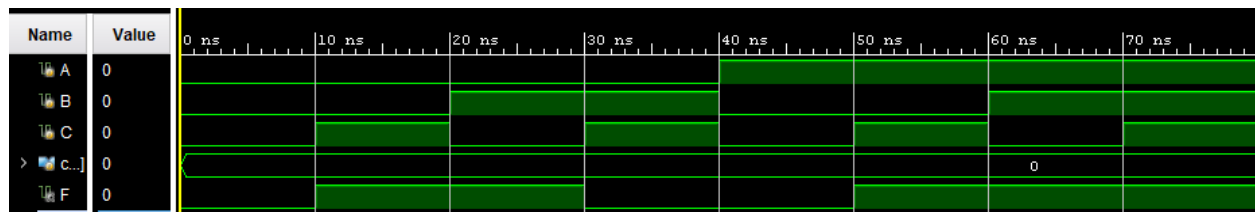
```
23 module lab3(  
24     input A, B, C,  
25     output F  
26 );  
27  
28     assign F = (~A)*(~B)*C + (~A)*B*(~C) + A*(~B)*C + A*B*(~C) + A*B*C;  
29 endmodule
```

AFTER SIMPLIFICATION:

```
23 module lab3(  
24     input A, B, C,  
25     output F  
26 );  
27  
28     assign F = (~A)*(~B)*C + (~A)*B*(~C) + A*(~B)*C + A*B;  
29 endmodule
```

- Screenshot of waveform

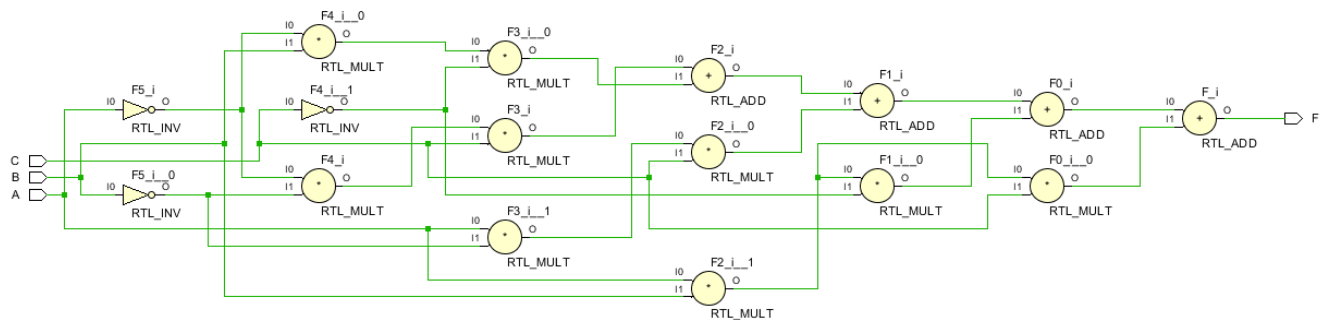
The waveform is the same for before and after the simplification and reflects the truth table given above. F is only 1 when (let the order be ABC), 001, 010, 101, 110, and 111; which are 5 terms.



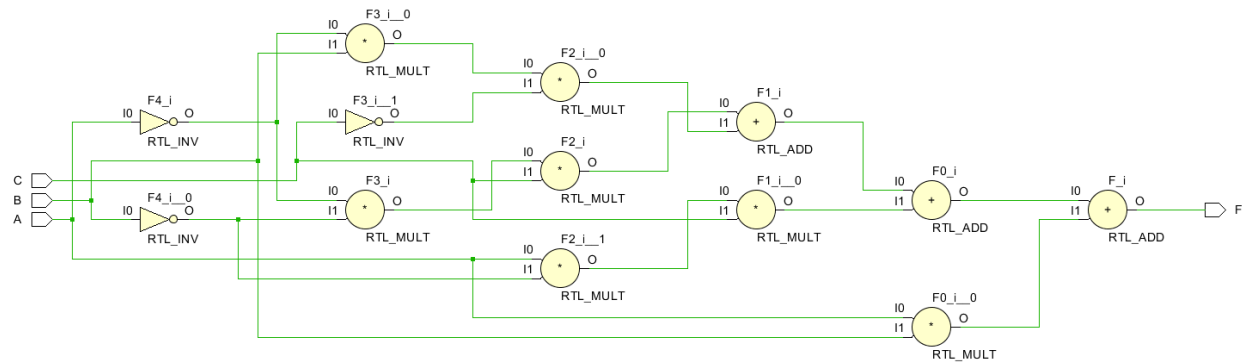
- Screenshot of schematic before & after simplification

The schematic before and after simplification is reflected below.

BEFORE:



AFTER SIMPLIFICATION:



- Simplified Boolean equation with steps

$$A'B'C + A'BC' + AB'C + \textcolor{red}{ABC'} + \textcolor{red}{ABC} \quad \text{Distributive Property}$$

$$A'B'C + A'BC' + AB'C + AB(\textcolor{red}{C'} + \textcolor{red}{C}): \quad C' + C = 1$$

$$\textcolor{yellow}{A'B'C + A'BC' + AB'C + AB}$$

Part 3: 4-input Boolean Circuit Simplification using Laws

Inputs				Outputs
A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

- Verilog RTL code

The source code involves 4 inputs(A,B,C, D), and 1 output(F). The code before simplification is the most maximized. The simplified equation was achieved with distributive property and inverse law. They each provide the same waveform, indicating that they're equal.

BEFORE:

```
22 //lab 3 part 3
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27
28     assign F = (~A) * (~B) * C * (~D) + (~A) * B * C * (~D) + (~A) * B * C * D + A * (~B) * C * (~D);
29 endmodule
30
```

AFTER SIMPLIFICATION:

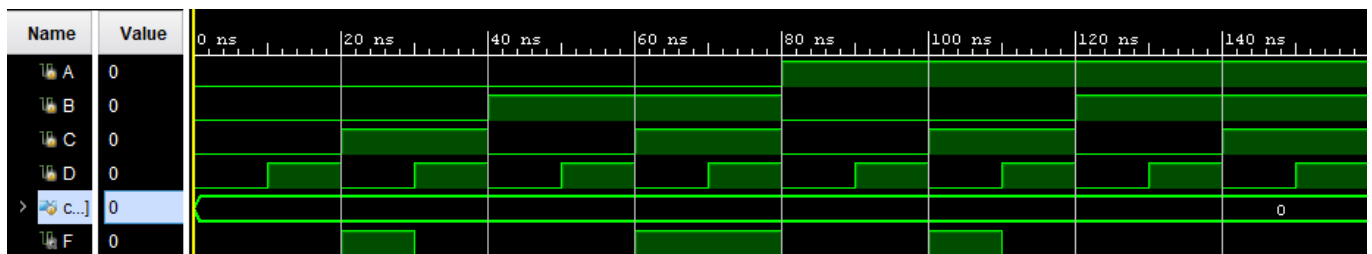
```

22 //lab 3 part 3
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27
28     assign F = (~A)*(~B)*C*(~D) + (~A)*B*C + A*(~B)*C*(~D);
29 endmodule
30

```

- Screenshot of waveform

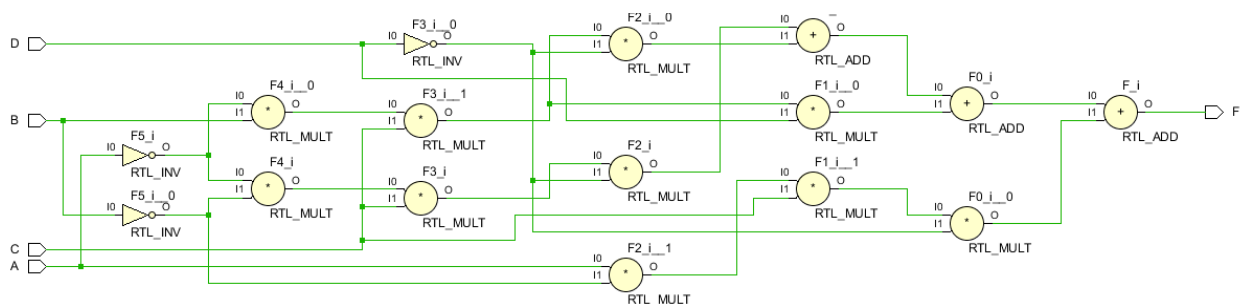
The waveform is the same for before and after the simplification and reflects the truth table given above. F is only 1 when (let the order be ABCD), 0010, 0110, 0111 and 1010; which are 4 terms.



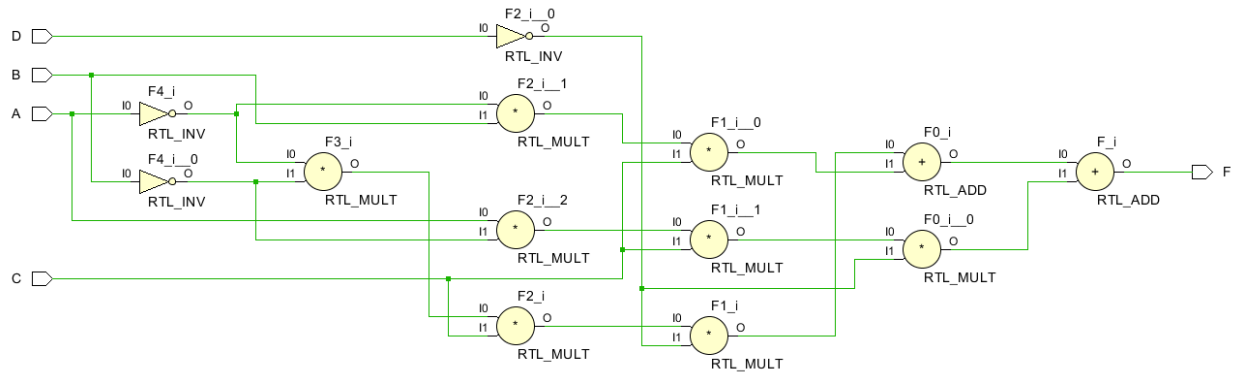
- Screenshot of schematic before & after simplification:

The schematic before and after simplification is reflected below.

BEFORE:



AFTER:



- Simplified Boolean equation with Steps

$$A'B'CD' + A'BCD' + A'BCD + AB'CD' \quad \text{Distributive}$$

$$A'B'CD' + A'BC(D' + D) + AB'CD' \quad D + D' = 1$$

$$A'B'CD' + A'BC + AB'CD'$$

Part 4: Boolean Simplification using K-Maps

Inputs				Outputs
A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

- Verilog RTL code

The source code involves 4 inputs(A,B,C, D), and 1 output(F). The code before simplification is the most maximized. The simplified equation through SOP was achieved with distributive property and inverse law. The simplest equation is given by kmaps. They each provide the same waveform, indicating that they're equal.

BEFORE:

```
22 //lab 3 part 4
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27
28     assign F = ~A*~B*C*~D + ~A*B*~C*D + ~A*B*C*~D + ~A*B*C*D + A*~B*C*~D + A*B*~C*~D + A*B*~C*D;
29 endmodule
30
```

AFTER SIMPLIFICATION FOR SOP:

```

22 //lab 3 part 4
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27
28     assign F = ~A*~B*C*~D + ~A*B*~C*D + ~A*B*C + A*~B*C*~D + A*B*~C*~D + A*B*~C*D;
29 endmodule

```

AFTER SIMPLIFICATION THROUGH K-MAPS:

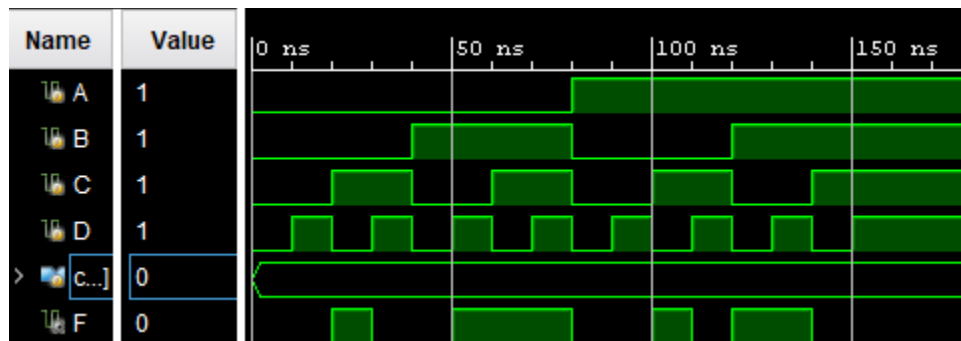
```

22 //lab 3 part 4
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27
28     assign F = A*B*(~C) + (~A)*C*(~D) + (~A)*B*D + A*(~B)*C*(~D);
29 endmodule

```

- Screenshot of waveform

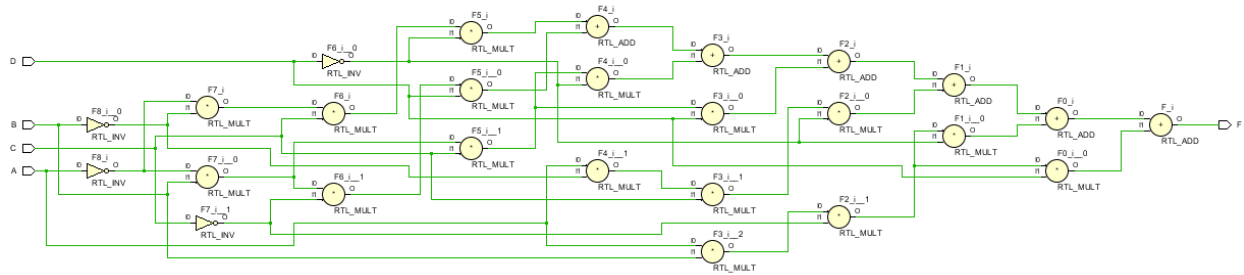
The waveform is the same for before and after the simplification through SOP and kmaps and reflects the truth table given above. F is only 1 when (let the order be ABCD), 0010, 0101, 0110, 0111, 1010, 1100, and 1101; which are 7 terms.



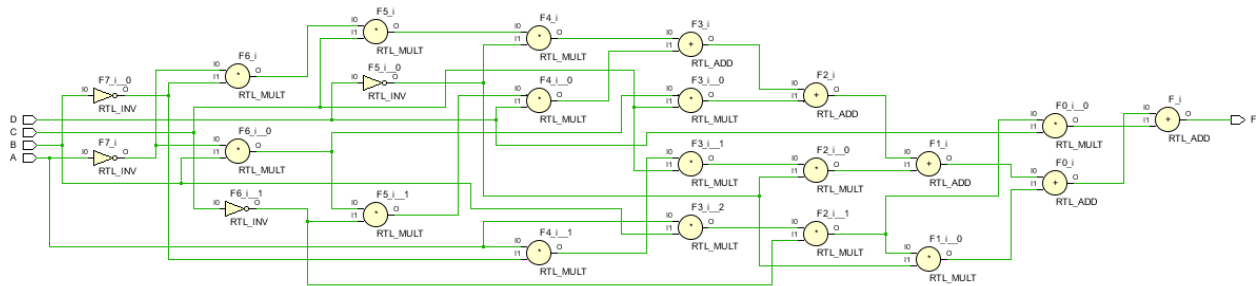
- Screenshot of schematic for SOP simplification

The schematic before and after simplification for SOP and kmaps is reflected below.

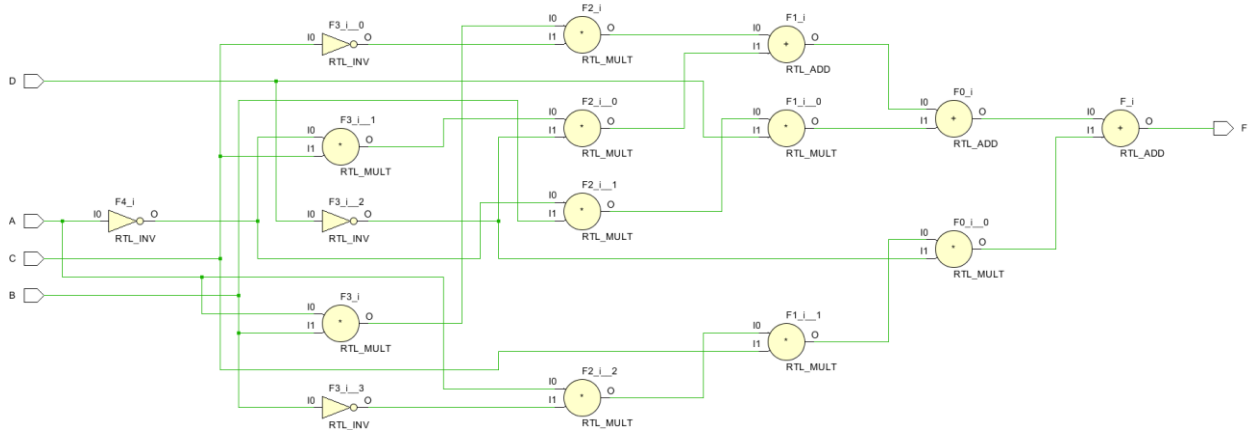
BEFORE:



AFTER SIMPLIFICATION FOR SOP:



AFTER SIMPLIFICATION THROUGH K-MAPS:



- Simplified Boolean equation for SOP

SOP: $A'B'CD' + A'BC'D + A'BCD' + A'BCD + AB'CD' + ABC'D' + ABC'D$ Distributive Property

$A'B'CD' + A'BC'D + A'BC(D'+D) + AB'CD' + ABC'D' + ABC'D$

$D'+D = 1$

$A'B'CD' + A'BC'D + A'BC + AB'CD' + ABC'D' + ABC'D$

- K-Map used to simplify the circuit

$$A'B'CD' + A'BC'D + A'BCD' + A'BCD + AB'CD' + ABC'D' + ABC'D$$

		AB			
		00	01	11	10
CD	00			1	
	01		1	1	
	11				
	10	1	1		1

$$0010 + 0101 + 0110 + 0111 + 1010 + 1100 + 1101$$

$$A=1, B=1, C=0, D=\cancel{10}$$

$$AB\bar{C}$$

$$A=0, B=\cancel{10}, C=1, D=0$$

$$A\bar{C}D'$$

$$A=0, B=1, C=\cancel{10}, D=1$$

$$A'BD$$

$$AB'CD'$$

$$F = AB\bar{C} + A\bar{C}D' + A'BD + AB'CD'$$

$$ABC' + A'CD' + A'BD + AB'CD'$$

Part 5: Application: Reduce Transistor Count for a Digital System

Output = 1 for 3,7,8,9,10,13

Output = 0 for 1,2,5,6,11,15

Output = x for 0,4,12,14 (Don't care)

- Verilog RTL code

The source code involves 4 inputs(A,B,C, D), and 1 output(F). The code before simplification is the most maximized. The simplified equation through SOP was achieved with distributive property and inverse law. The simplest equation is given by kmaps. They each provide the same waveform, indicating that they're equal.

BEFORE

```
22 //lab 3 part 5
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27     assign F = ~A*~B*C*D + ~A*B*C*D + A*~B*~C*~D + A*~B*~C*D + A*~B*C*~D + A*B*~C*D;
28
29 endmodule
```

AFTER SIMPLIFICATION FOR SOP:

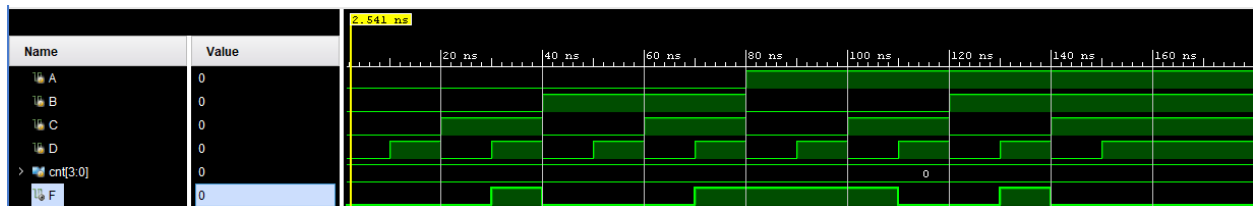
```
22 //lab 3 part 5
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27     assign F = ~A*~B*C*D + ~A*B*C*D + A*~B*~C + A*~B*C*~D + A*B*~C*D;
28
29 endmodule
```

AFTER SIMPLIFICATION THROUGH KMAP

```
22 //lab 3 part 5
23 module lab3(
24     input A, B, C, D,
25     output F
26 );
27     assign F = ~A*C*D + A*~C*D + A*~B*~D;
28 endmodule
```

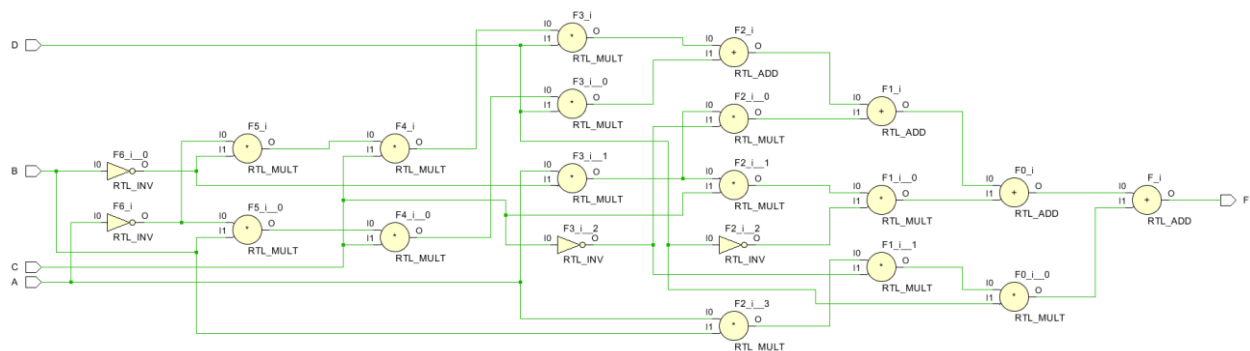
- Screenshot of waveform

The waveform is the same for before and after the simplification through SOP and kmaps and reflects the truth table given above. F is only 1 when (let the order be ABCD), 0011, 0111, 1000, 1001, 1010, and 1101; which are 6 terms.



- Screenshot of schematic for SOP simplification

The schematic after simplification through SOP is reflected below.



- Simplified Boolean equation for SOP

$$A'B'CD + A'BCD + AB'C'D' + AB'C'D + AB'CD' + ABC'D$$

Distributive Property

$$A'B'CD + A'BCD + AB'C'(D'+D) + AB'CD' + ABC'D$$

$$D'+D = 1$$

$$A'B'CD + A'BCD + AB'C' + AB'CD' + ABC'D$$

- K-Map used to simplify the circuit

part 5:

Output = 1 for 3, 7, 8, 9, 10, 13

Sum of Products = (Output = 1)

$0011 + 0111 + 1000 + 1001 + 1010 + 1101$

$A'B'CD + A'BCD + AB'C'D + AB'CD + ABC'D + ABC'D$

Don't care: 0, 4, 12, 14

$0000 + 0100 + 1100 + 1110$

AB \ CD	00	01	11	10
00	X	X	X	1
01			1	1
11	1	1		
10			X	1

• $A=0, B=1, C=1, D=1: A'CD$

• $A=1, B=1, C=0, D=1: AC'D$

• $A=1, B=0, C=0, D=0: AB'D'$

$F = A'CD + AC'D + AB'D'$

Q&A:

- What form of Boolean equations is an ANDs of ORs?

Also Product of Sums, is in the form of $(A+B)(C+D)(E+F)$

- What form of Boolean equations is an ORs of ANDs?

Also Sum of Products, is in the form of $AB+CD+EF$

- How many logic gates comprise a 3 variable term?

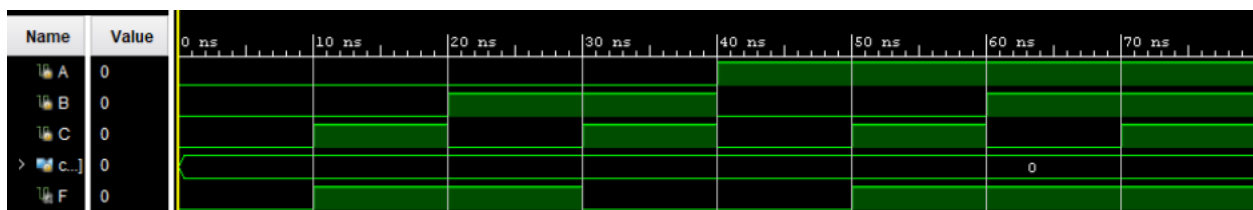
The number of logic gates of 3 variables can reach up to 8 gates.

- What does a logic gate represent within the physical realm of computers?

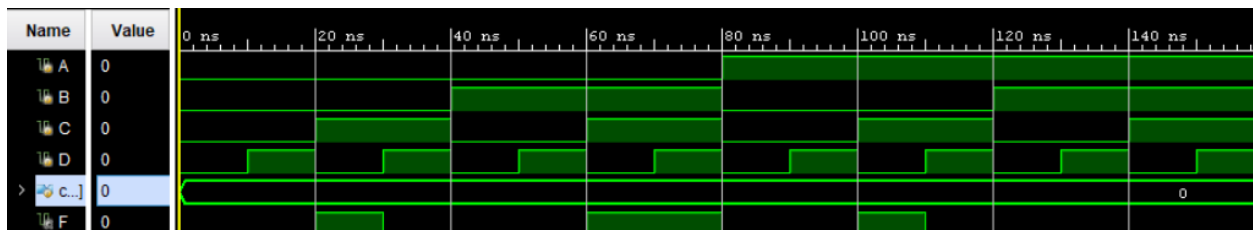
A logic gate in the physical realm of computers are transistors that act as switches between 1s and 0s.

Test Plan:

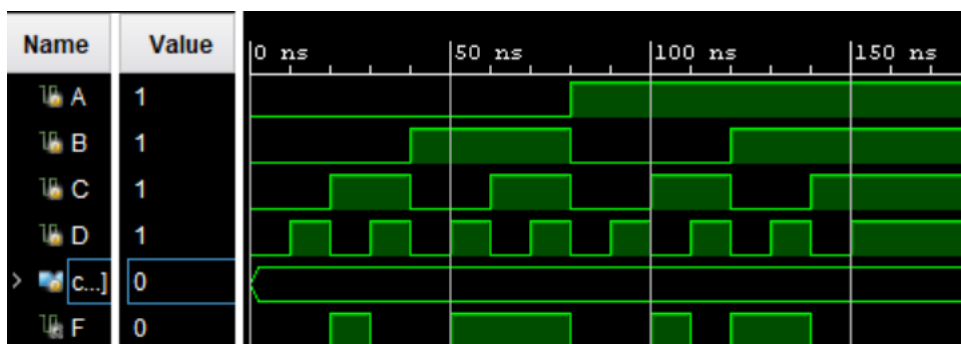
First, I coded a simulation that runs through every possible input combinations, dependent on the number of inputs. Next, for parts 2-4, I checked if the waveforms matched the truth tables, including their values. For part 5, I ensured I inputted the correct Boolean equation into the source code. After verifying the tests, I recorded the waveforms and data alongside them.



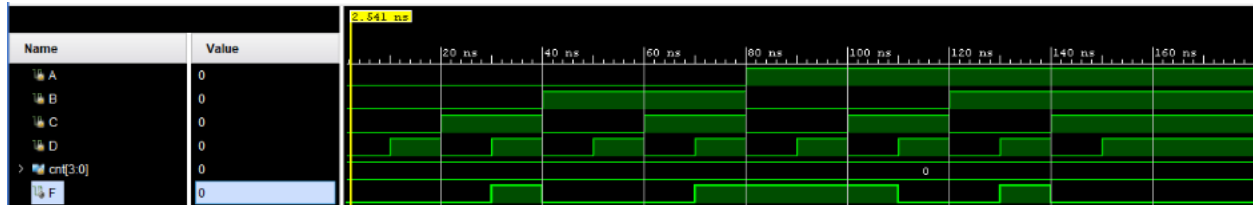
Part 2: Test Bench for 3-input, 1 output,



Part 3: Test Bench for 4-input, 1 output



Part 4: Test Bench for 4-input, 1 output



Part 5: Test Bench for 4-input, 1 output

Conclusion:

This experiment was a success. The results for part 2-5, were as expected after coding in the test bench. I had reached a roadblock for part 4 initially, where I mislabeled my kmaps and received a different response. This I was able to correct by redoing the kmap in a different orientation, in order to not fall into the same mindset. The use of Boolean equations in this software has proven to have real world applications and has furthered understanding of simplifying Boolean equations for more efficient algorithms.